RSNCPU 研究计划 2026-2027

研究对象： 自动驾驶场景中的二维视觉目标检测任务，重点关注车辆、行人、骑行者以及交通标志、交通灯等典型道路参与者与交通要素的检测与识别

应用场景与约束： 面向边缘侧部署场景，考虑车载平台在计算资源、片上存储、功耗预算和实时响应等方面的实际约束，关注目标检测算法在资源受限条件下的可部署性与运行效率

拟解决的问题

1. **端到端任务中的数据通信冗余**：在传统异构架构中，CPU 与加速器物理隔离，数据需通过系统总线及 DMA 在不同存储层级之间频繁搬运。已有权威研究表明，DNN 系统的能耗与时延不仅由计算量决定，数据搬运与存储访问同样是关键瓶颈 [8][9]。进一步地，在端到端机器学习任务中，除 DNN 计算本身外，预处理、后处理与数据管理会消耗大量延迟和功耗；在典型边缘视觉应用中，预处理开销甚至可超过总运行时间的 70%，而预/后处理与数据管理整体可占总运行时间约 60% [7][10][11]。因此，这类 CPU 与加速器分离的异构组织方式容易产生显著的数据通信冗余与无效能耗。

2. **异构计算资源的低效协同**：边缘设备中，CPU 性能有限，常导致预处理、层间处理和后处理阶段成为整体流程的短板。已有研究表明，常见 NPU/脉动阵列加速器主要负责神经网络层中的 MAC 计算，而诸如 padding、batch normalization、数据对齐、数据复制等层间数据准备工作仍需依赖 CPU 核心协同完成，并要求高效的核间通信与数据搬运支持 [7]。与此同时，边缘视觉 SoC 在从传感器输入、图像信号处理到推理输出的完整链路中，通常还需要额外的预处理、结果筛选和后处理模块协同工作 [10][11]。当 CPU 或通用数字处理模块承担这些复杂数据处理任务时，高性能加速器容易处于等待或空闲状态，资源难以在任务流中动态复用，进而导致整体系统利用率和能效下降。

3. **固定架构与目标检测模型之间的匹配矛盾**：传统固定尺寸脉动阵列对规则、致密的大规模矩阵乘法具有较高计算效率 [1]。然而，自动驾驶目标检测模型通常包含多尺度特征图、分支结构和跨层特征融合，不同网络层在计算规模、数据复用方式和带宽需求上存在显著差异 [4][5][6]。已有研究表明，面向大规模规则 DNN 优化的固定数据流加速器在面对层形状与尺寸变化较大的模型时，容易出现 PE 利用率下降，并导致性能和能效下降 [2][3]。因此，在边缘部署场景下，固定架构与目标检测模型之间存在明显的映射失配风险。

创新点

1. **架构创新**：针对边缘计算场景下自动驾驶目标检测任务的端到端推理需求，以 SNCPU 的可重构统一架构为基础，采用权重固定（WS）双向脉动数据流保持高数据局部性，同时引入 Gemmini 风格的细粒度自定义 RISC-V ISA 实现灵活的加速器控制，支持多模式切换，解决端到端任务中的数据通信冗余和异构计算资源低效协同问题，预期实现高数据局部性和高资源利用率。

2. **任务导向的算法-硬件协同创新**：面向 YOLOv11n INT8 目标检测模型，研究动态任务感知的阵列切分与片上数据流组织方法，根据网络层计算特征以及预处理、层间处理和后处理负载动态分配资源，在权重固定数据流基础上实现灵活计算映射与任务级流水。该设计主要针对目标检测模型多尺度、多分支结构带来的系统级资源失配问题 [4][5][6]，并参考已有关于灵活数据流和利用率优化的研究思路 [2][3]，预期在一定程度上缓解固定架构与目标检测任务之间的匹配矛盾，降低端到端推理延时。

研究目标（Goal）

> 实现 RSNCPU 处理器（基于 SNCPU 架构，Chisel 实现），以 `YOLOv11n INT8` 作为核心验证模型，在 `Xilinx ZCU102 FPGA` 平台上完成自动驾驶场景二维视觉目标检测任务的高效部署与验证。

本研究面向自动驾驶场景中的二维视觉目标检测任务，围绕车辆、行人、骑行者以及交通标志、交通灯等典型目标，研究适用于边缘侧部署的高效推理与硬件加速实现方法。以 `YOLOv11n INT8` 为主要算法载体，在 `BDD100K` 自动驾驶场景数据集上对检测精度进行验证，在保证关键检测精度指标受控的前提下，重点降低系统推理延时、片上数据搬运开销与整体能耗，并在 `ZCU102 FPGA` 平台完成功能、性能与可实现性验证。

RSNCPU 目标参数（Xilinx ZCU102 FPGA）：

参数	RSNCPU 目标值	其他工作（Gemmini / SNCPU）
PE 阵列规模	10×10，INT8，WS 数据流	SNCPU：10×10 可重构阵列；Gemmini：可配置 2-D 阵列 [7][12]
目标频率	200MHz	SNCPU（65nm ASIC）：400MHz；Gemmini：依配置与工艺而定 [7][12]
峰值算力 (INT8)	40 GOPS（100 PEs × 2 ops/MAC × 200MHz）	SNCPU：80 GOPS @ 400MHz；Gemmini：随阵列规模变化 [7][12]
预期 PE 利用率	DNN 模式 >=95%，CPU 模式 >=90%	SNCPU：DNN ~99%，CPU ~96%，整体 97%；Gemmini：未统一报告 [7]
核心验证模型	YOLOv11n INT8	SNCPU：VGG16 / ResNet18 / ELU；Gemmini：多类 DNN benchmark [7][12]
目标应用任务	自动驾驶场景二维视觉目标检测	SNCPU：端到端图像分类；Gemmini：异构 DNN 推理 [7][12]
YOLOv11n 纯 DNN 计算延时	~170ms（单张 640×640 图片，~6.5 GOPs）	公开文献未见与本配置直接可比的 YOLOv11n 结果
YOLOv11n 端到端推	~180-190ms	SNCPU 相对 Gemmini 端到端延时改善 39%–64% [7]

参数	RSNCPU 目标值	其他工作（Gemmini / SNCPU）
理延时（RSNCPU）		
同算力传统异构架构端到端延时（Baseline）	~250-270ms	Gemmini（Rocket+RoCC+DMA 基线）[7][12]
相对传统架构端到端改善	>=30%（SNCPU 论文报告 39%-64%）	SNCPU vs Gemmini：39%–64% [7]

说明：硬件参数对比中，`Gemmini` 用于代表传统 CPU+RoCC 加速器分离式异构基线，`SNCPU` 用于代表与 RSNCPU 架构思想最接近的统一可重构参考设计。

软件目标参数（YOLOv11n INT8，BDD100K）：

参数	RSNCPU 目标值	其他工作（同类轻量级参考工作 / 公开较高精度参考结果）
自动驾驶场景验证数据集	BDD100K validation set	SES-YOLOv8n：BDD100K [13]；DDRN-RN50：BDD100K [14]
核心检测类别	car / pedestrian / rider	BDD100K 道路目标检测任务，覆盖车辆、行人、骑行者等关键类别 [13][14]
BDD100K val AP (0.5:0.95)	>=25	SES-YOLOv8n：22.3 [13]；DDRN-RN50：30.2 [14]
BDD100K val AP50	>=48	SES-YOLOv8n：41.9 [13]；DDRN-RN50：54.9 [14]
BDD100	>=35	BDD100K 公开论文通常未统一报告该指标

参数	RSNCPU 目标值	其他工作（同类轻量级参考工作 / 公开较高精度参考结果）
K val AR_max_100		
关键类别平均精确率（IoU=0.50，conf=0.25）	>=75%	BDD100K 公开论文通常未统一报告该口径指标
关键类别平均召回率（IoU=0.50，conf=0.25）	>=70%	BDD100K 公开论文通常未统一报告该口径指标
INT8 相对 FP32 的 AP50 精度损失	<=2 个百分点	YOLOv8 边缘 INT8 量化案例：约 1.6 个百分点 [15]
INT8 相对 FP32 的关键类别平均召回率下	<=2 个百分点	公开 BDD100K 量化论文未统一报告该指标

参数	RSNCPU 目标值	其他工作（同类轻量级参考工作 / 公开较高精度参考结果）
降		

说明：软件参数对比中，文献[13] 作为同类轻量级参考工作，用于反映轻量化自动驾驶检测模型在 BDD100K 上的可实现水平；文献[14] 作为公开较高精度参考结果，用于表征当前公开方法在同数据集上的较优精度区间。

SNCPU 参考参数（65nm ASIC）：

参数	值
频率	400MHz
面积	4.47mm²
SRAM	150kB
最低电压	0.5V
能效	655 GOPS/W（1.0V）至 1.8 TOPS/W（0.5V）
PE 利用率	DNN 模式 ~99%，CPU 模式 ~96%，整体平均 97%

架构方案（Plan B - 软硬件并行 MVP 路线）： 以 SNCPU 论文为架构蓝本，以 Gemmini PE 的 MAC 单元（`MacUnit` + `c1/c2` 寄存器）为 Chisel 实现基础，在其上叠加 SNCPU 论文所描述的位置相关 CPU 流水线阶段逻辑（流水线阶段模板 + 按 PE 位置分配），构建可在脉动阵列加速模式与 RISC-V CPU 模式之间动态切换的可重构 PE 阵列。Gemmini 的 Chisel/Chipyard 生态、Scratchpad 寻址机制和解耦访问/执行架构为设计提供实现框架参考。

技术选型： - 硬件描述语言：Chisel/Scala（与 Gemmini 保持一致） - FPGA 目标：Xilinx ZCU102（XCZU9EG-FFVB1156，600K LUT，ARM A53 四核） - 推理精度：INT8 量化 - 核心验证模型：YOLOv11n（~2.6M 参数，优先） - 目标任务：自动驾驶场景二维视觉目标检测 - 答辩截止：2027 年 3 月

关键架构理解（SNCPU 核心设计）

PE 阵列结构

- 10×10 可重构 PE 阵列
- 每**行**的 10 个 PE = 1 个完整 5 级 RISC-V CPU 流水线（行 CPU 模式）
- 每**列**的 10 个 PE = 1 个完整 5 级 RISC-V CPU 流水线（列 CPU 模式）
- 同一 PE 同时支持：行 CPU 的第 f(col) 流水线阶段 + 列 CPU 的第 g(row) 流水线阶段 + DNN MAC
- PE 设计是**位置相关（非均匀）**的：位于 (row_i, col_j) 的 PE 需同时支持两种 CPU 角色 + MAC

流水线阶段到 PE 的映射（以行为例，10 个 PE）

PE 位置	CPU 流水线阶段	复用 MAC 组件	逻辑复用率
PE1	PC（程序计数器）	MAC 加法器 + 32-bit 寄存器	69%
PE2-3	IF（取指）	32-bit 内部寄存器（c1/c2）+ 8-bit 输入寄存器	80%
PE4-5	ID（译码）	8-bit 乘法器 + 32-bit 加法器重构为操作码/功能码解码器	80%
PE6-7-8	EX（执行）	PE6=ALU+布尔+移位器，PE7=分支地址计算，PE8=寄存器透传	77%
PE9	MEM（访存）	ALU 结果到/从 Dcache	64%
PE10	WB（写回）	Dcache→寄存器堆，MUX 逻辑	64%

整体面积开销：9.8%（PE 阵列 3.4% + MEM 6.4%）

五种工作模式

模式	AOMEM 角色	数据流向
列加速器模式	行 AOMEM = 激活输入，底部 = 累加输出	数据从上到下
行 CPU 模式	右侧 AOMEM = 数据缓存，左侧指令缓存激活	指令从左到右流水
行加速器模式	底部 AOMEM = 输入，右侧 = 输出	数据从左到右
列 CPU 模式	底部 AOMEM = 数据缓存，顶部指令缓存激活	指令从上到下流水
混合模式	部分行/列为加速器 AOMEM + 部分为 CPU Dcache	按子阵列划分独立运行

混合模式说明（SNCPU 论文 Fig.5(c)）：50% PE 配置为 DNN 加速器、50% 配置为 5 核 RISC-V CPU，为不均衡 CPU/DNN 工作负载提供灵活分配。此模式是动态任务感知阵列切分的基础。

四步双向数据流（彻底消除 DMA）

- Step1: 行 CPU 模式（图像预处理）
- Step2: 列加速器模式（DNN 第 1 层推理）
- Step3: 列 CPU 模式（层间处理）
- Step4: 行加速器模式（DNN 第 2 层推理）
- 中间数据全程保留在 AOMEM，无需 DMA 搬运

自定义 ISA（4 条指令）

指令	功能	时序
ASI	保存 CSR 参数（输入向量数、输出向量数、缩放系数、conv/FC 选择）	6 周期
MSI	模式切换（CPU→DNN：13-15 周期；DNN→CPU：3 周期）	—
SL2	多核 CPU 间通过 L2 SRAM 写入通信	—
LL2	多核 CPU 间通过 L2 SRAM 读取通信	—

ACT 模块（每行/列 DNN 后处理单元）

- 每行/列配备 1 个 ACT 模块，负责 DNN 后处理功能
- 功能包括：部分和累积、池化（max/avg）、ReLU 激活、输出缩放
- 与 AOMEM 协同工作：ACT 处理完成后数据直接写回 AOMEM，无需额外搬运
- 在 CPU 模式下 ACT 模块通过时钟门控关闭以节省功耗

完整时间规划

阶段 0：基础研究与微架构设计（2026.03–04，2 个月）

任务：

2026.03 — Gemmini RTL 精读 + SNCPU 论文精读与 PE 映射

> **阅读顺序说明**：先精读 Gemmini RTL（已有项目基础，学习阻力最小），建立具体的 Chisel 代码级实现认知后，再读 SNCPU 论文的 PE 逻辑复用方案，可以立刻判断哪些组件能直接复用、哪些需要新增，理解效率更高。

1. **Gemmini RTL 精读**（正式任务，非可选，**先执行**）
    - `PE.scala`：`MacUnit` 内部结构、`c1/c2` 寄存器机制、`out_b` 数据路径（RSNCPU PE 直接改造基础）
    - `Tile.scala`：移位寄存器打拍方式（RSNCPU 流水线寄存器参考）
    - `Mesh.scala`：阵列互连与数据流实现参考
    - `Scratchpad.scala`：SRAM bank 寻址机制（AOMEM 设计参考）
    - **带着问题读**（读 RTL 时记录，读 SNCPU 论文时对照解答）：
        - `MacUnit` 的加法器和 32-bit 寄存器如何被 SNCPU 复用为 PC?
        - `c1/c2` 寄存器如何复用为 IF 阶段的指令寄存器?
        - `Scratchpad` 的 bank 寻址机制与 AOMEM 三角色切换有何异同?
2. **SNCPU 论文精读**（Section II-V，含芯片实测数据，**后执行**）

    - 完成 PE 位置映射表：每个 (row_i, col_j) PE 对应的行 CPU 流水线阶段 + 列 CPU 流水线阶段
    - 整理 ASI/MSI/SL2/LL2 指令语义与时序要求
    - 理解五种工作模式的数据路由与 AOMEM 重配置机制
    - 理解 ACT 模块在每行/列中的角色（部分和累积、池化、ReLU、缩放）
    - **对照 Gemmini RTL 回答前述问题**，记录可复用组件与需新增逻辑的初步清单

2026.04 — 微架构设计 + YOLOv11n 基准

3. **微架构设计文档 v1.0**
    - 基于以上两项，写出 RSNCPU PE 模板的输入/输出端口定义
    - 确定 MAC 复用策略（哪些逻辑直接复用 Gemmini `MacUnit`，哪些新增）
    - 设计流水线阶段模板分配方案：定义 (row_i, col_j) → （行流水线阶段，列流水线阶段）的映射规则
    - 设计 AOMEM 重配置接口（激活输入/输出/Dcache 三角色切换）
4. **并行：YOLOv11n INT8 量化启动**
    - 在 x86 上用 ultralytics 工具链导出 YOLOv11n INT8 量化权重（.bin 格式），提取 scale/zero-point 参数表
    - 在 **Spike + Gemmini** 上用 C 代码运行 YOLOv11n INT8 推理功能验证（参考现有 `conv.c/conv_dw.c` 框架，调用 `gemmini_loop_conv_ws`），作为后续移植到 RSNCPU 的功能基准
    - 新建 `shen_yolov11n_gemmini.c` 完成端到端推理验证，记录各层输出数值

里程碑： PE 位置映射表完成；微架构设计文档 v1.0（含 PE 模板端口定义 + AOMEM 接口）完成；Spike+Gemmini 上 YOLOv11n INT8 推理功能验证通过

阶段 1：可重构 PE 模板 RTL 实现（2026.05-07，3 个月）

核心挑战：每个 PE 需同时支持 DNN MAC + 行 CPU 流水线阶段 + 列 CPU 流水线阶段，设计需位置相关。采用 SNCPU 论文的模板化方法：先创建流水线阶段模板，再根据 PE 在二维阵列中的位置分配具体模板。

2026.05 — PC/IF + ID 阶段模板

- 设计 `shen_ReconfigPE` Chisel 模块基础框架（端口、模式选择信号）
- 实现 PC 阶段逻辑（复用 MAC 加法器，寄存器宽度适配 32-bit）
- 实现 IF 阶段逻辑（复用 c1/c2 寄存器作为指令寄存器）
- 实现 ID 阶段逻辑（用 8-bit 乘法器 + 32-bit 加法器重构为操作码/功能码解码器）
- 单 PE 仿真验证（DNN 模式 + PC/IF + ID 模式切换正确性）
- 参考文件：`src/main/scala/gemmini/PE.scala`

2026.06 — EX + MEM + WB 阶段模板

- 实现 EX 阶段（PE6=ALU+布尔+移位器，PE7=分支地址计算，PE8=寄存器透传）

- 实现 MEM 阶段（ALU 结果与 Dcache 接口）
- 实现 WB 阶段（Dcache→RF MUX 逻辑）
- 全部 5 类流水线阶段模板单元仿真通过
- 参考文件：`src/main/scala/gemmini/Tile.scala`

2026.07 — 1 行 × 10 PE = 完整 RISC-V CPU 集成仿真

- 将 10 个 `shen_ReconfigPE` 按流水线顺序连接，实现行 CPU 模式数据通路
- 编写 RV32I 指令集仿真测试（至少覆盖：加减法、分支跳转、load/store、立即数）
- 同时验证 DNN 模式（10 个 PE 作为脉动阵列一行）

里程碑： RV32I 指令集仿真测试通过；Spike+Gemmini 上 YOLOv11n INT8 推理基准数据就绪

阶段 2：AOMEM + ACT 模块 + 五种工作模式（2026.08-09，2 个月）

2026.08 — AOMEM 模块 + ACT 模块 + 5 种数据路由

- 实现 `shen_AOMEM` Chisel 模块（3 角色热配置：激活输入 SRAM / 输出 SRAM / 数据缓存）
- 实现 `shen_ACTUnit` Chisel 模块：部分和累积、池化（max/avg）、ReLU、输出缩放
- 实现五种工作模式的顶层数据路由逻辑（模式控制信号、互连开关）
  - 列加速器模式、行 CPU 模式、行加速器模式、列 CPU 模式、混合模式
- 5 种模式独立仿真验证

2026.09 — 四步双向数据流端到端仿真

- 实现四步双向数据流控制器（Step1→2→3→4 状态机）
- 验证：小规模 CNN（如 2 层 conv）全程无 DMA 从 AOMEM 流转
- 测量中间数据的 SRAM 占用，验证无外部搬运

里程碑： 小规模 CNN 四步双向数据流端到端仿真通过（零 DMA）；混合模式仿真通过

阶段 3：顶层控制器 + 自定义 ISA + 多核同步（2026.10）

- 实现顶层 `shen_RSNCPUController` Chisel 模块（集成 PE 阵列、AOMEM、ACT、模式控制）
- 实现 4 条自定义指令的硬件解码逻辑：
  - ASI：CSR 参数配置（6 周期）
  - MSI：模式切换（CPU→DNN 13-15 周期，DNN→CPU 3 周期），含方向设置、AOMEM 配置、初始值设置
  - SL2/LL2：多核 CPU 间通过 L2 SRAM bank 的写入/读取通信
- 实现多核 CPU 同步机制：
  - 每行/列小型控制器（SRAM 数据仲裁 + 模式切换协调）
  - 2 个 L2 SRAM bank 用于核间数据共享（CPU 模式专用）

    - 多核并行任务分配策略（如图像预处理的 10 核并行分块）
- 扩展 Spike 功能模型：添加 ASI、MSI、SL2、LL2 指令解码
- 扩展 C 测试框架：新增 `shen_rsncpu.h` 头文件封装上述指令
- 在 Spike 上运行 YOLOv11n INT8 推理（功能正确性验证）

里程碑： Spike 上 YOLOv11n 推理通过（功能正确）；多核 CPU SL2/LL2 通信验证通过

阶段 4：10×10 全系统集成 RTL 仿真（2026.11）

- 完整 10×10 PE 阵列集成（`shen_Mesh10x10`）
- 顶层系统集成（PE 阵列 + AOMEM × 4 侧 + ACT × 20 + 控制器 + 指令缓存接口 + L2 SRAM）
- YOLOv11n INT8 全网络 RTL 仿真（Verilator），记录推理延迟、带宽、PE 利用率
- 性能数据收集：延迟（ms）、吞吐量（TOPS）、能效（TOPS/W，估算）

里程碑： YOLOv11n RTL 仿真性能数据完整

阶段 5：ZCU102 FPGA 综合与演示（2026.12–2027.01，2 个月）

2026.12 — FPGA 综合与时序收敛

- 使用 Vivado 综合 RSNCPU RTL，目标频率 200MHz（ZCU102 保守目标）
- 解决时序违例（关键路径：PE 内 MAC ↔ CPU 逻辑复用的多路选择器）
- 资源利用率分析（LUT、BRAM、DSP48）

2027.01 — FPGA 演示系统

- 集成 ZCU102 ARM A53 主机软件（摄像头输入 / 图像预处理）
- 实现主机通过 AXI 接口驱动 RSNCPU FPGA 加速器
- 端到端演示：摄像头图像 → RSNCPU FPGA 推理 → YOLOv11n 目标检测框输出

里程碑： FPGA 实物运行 YOLOv11n 目标检测演示

阶段 6：实验数据补充 + 论文撰写与答辩（2027.02–03，2 个月）

2027.02 — 实验数据补充 + 论文初稿

- 补充消融实验：有/无四步双向数据流的 DMA 开销对比
- 补充对比实验：与 Gemmini（纯加速器）在 YOLOv11n 上的性能/能效对比
- 补充混合模式 vs 纯 DNN 模式的 PE 利用率对比
- 整理完整实验数据表格
- 论文初稿撰写（2027.02 月中前完成）

2027.03 — 论文修改与答辩

- 导师审阅修改（2027.03 上旬）

- 答辩准备（PPT、演讲稿、问答预演）
- 正式答辩（2027.03）

里程碑： 通过硕士论文答辩

总时间轴一览

月份	阶段	主要任务	关键里程碑
2026.03	阶段 0a	Gemmini RTL 精读（先）+ SNCPU 论文精读（后）与 PE 映射	PE 位置映射表完成
2026.04	阶段 0b	微架构设计文档 v1.0 + YOLOv11n INT8 量化基准	微架构文档 v1.0；YOLOv11n Spike 推理通过
2026.05	阶段 1a	PC/IF + ID 阶段模板 RTL	单 PE 模式切换仿真通过
2026.06	阶段 1b	EX + MEM + WB 阶段模板 RTL	全部 5 类模板仿真通过
2026.07	阶段 1c	1 行 × 10 PE = 完整 CPU 集成仿真	RV32I 测试通过
2026.08	阶段 2a	AOMEM + ACT 模块 + 5 种工作模式	5 模式仿真通过
2026.09	阶段 2b	四步双向数据流端到端仿真	小规模 CNN 零 DMA 验证
2026.10	阶段 3	顶层控制器 + ASI/MSI/SL2/LL2 + 多核同步 + Spike 扩展	Spike 上 YOLOv11n 推理通过；多核通信验证
2026.11	阶段 4	10×10 全系统 RTL 仿真	YOLOv11n RTL 性能数据
2026.12	阶段 5a	ZCU102 FPGA 综合时序收敛	200MHz 时序通过
2027.01	阶段 5b	FPGA 演示系统	实物目标检测演示
2027.02	阶段 6a	实验数据补充 + 论文初稿	初稿完成；实验数据完整
2027.03	阶段 6b	论文修改 + 答辩	通过答辩

YOLOv11n 验证递进线索

阶段	验证环境	验证目标
阶段 0（04 月）	x86 + Spike/Gemmini	INT8 量化精度确认 + Gemmini 功能基准
阶段 3（10 月）	Spike/RSNCPU	RSNCPU 自定义 ISA 下的功能

阶段	验证环境	验证目标
		正确性
阶段 4（11 月）	Verilator RTL	周期精确性能数据（延迟、吞吐、PE 利用率）
阶段 5（01 月）	ZCU102 FPGA	实物端到端演示

风险矩阵

风险	概率	影响	缓解措施
PE 位置相关设计复杂度超预期	高	高	阶段 1 已留 3 个月缓冲；优先完成 1 行验证
AOMEM 三角色切换时序复杂	中	高	阶段 0 微架构设计提前定义接口；参考 Gemmini Scratchpad 寻址策略
FPGA 时序收敛困难	中	高	目标频率保守设定 200MHz；关键路径提前分析
YOLOv11n INT8 精度下降不可接受	低	中	提前在 x86 验证量化精度；备选 YOLOv11s
SNCPU 论文未披露实现细节	中	中	参考 Gemmini RTL 填补空白；记录设计决策
多核 CPU 同步调试耗时长	中	中	简化初期同步策略（先 barrier 同步，后优化）；SL2/LL2 综合测试提前
计划延迟压缩论文时间	中	高	论文架构可从阶段 4 并行准备；实验数据边测边记录

RSNCPU 相对 SNCPU 的创新点（答辩时重点强调）

1. **Gemmini 风格 ISA 扩展**：在 Chipyard/RoCC 接口框架下实现 ASI/MSI/SL2/LL2，与开源生态兼容
2. **混合模式支持**：支持不对称子阵列切分（如 30% CPU + 70% DNN），适配不同模型/任务负载比例
3. **YOLOv11n INT8 端到端验证**：在类 SNCPU 架构上验证 YOLOv11n INT8 在自动驾驶二维视觉目标检测场景中的边缘部署可行性
4. **FPGA 原型验证**：在 ZCU102 上实现实物演示，SNCPU 原文仅有 65nm ASIC 流片

后续优化方向（基础功能完成后可探索）

1. **功耗门控优化**：参考 SNCPU ~40% DNN 模式功耗节省，对 CPU-only SRAM、未使用触发器、输入逻辑等实施时钟门控

2. **动态任务感知阵列切分**：运行时根据模型层参数自动决定 CPU/DNN 切分比例
3. **稀疏优化**：参考 Eyeriss 的 RLC 压缩与数据门控，跳过零值计算
4. **多核 CPU 并行预处理优化**：参考 EdgeAI 的 OpenMP 并行框架

参考文献

一、体系结构与数据流优化

[8] Sze V, Chen Y H, Yang T J, Emer J S. Efficient Processing of Deep Neural Networks: A Tutorial and Survey[J]. Proceedings of the IEEE, 2017, 105(12): 2295-2329.

[1] Jouppi N P, Young C, Patil N, et al. In-Datacenter Performance Analysis of a Tensor Processing Unit[C]//Proceedings of the 44th Annual International Symposium on Computer Architecture. 2017.

[9] Chen Y H, Emer J, Sze V. Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks[J]. IEEE Journal of Solid-State Circuits, 2017, 52(1): 127-138.

[2] Chen Y H, Yang T J, Emer J, et al. Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices[J]. IEEE Journal on Emerging and Selected Topics in Circuits and Systems, 2019, 9(2): 292-308.

[3] Jha N K, Ravishankar S, Mittal S, et al. DRACO: Co-Optimizing Hardware Utilization, and Performance of DNNs on Systolic Accelerator[EB/OL]. arXiv:2006.15103, 2020.

二、目标检测模型与多尺度特征

[4] Lin T Y, Dollar P, Girshick R, et al. Feature Pyramid Networks for Object Detection[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2017: 2117-2125.

[5] Redmon J, Farhadi A. YOLOv3: An Incremental Improvement[EB/OL]. arXiv:1804.02767, 2018.

[6] Tan M, Pang R, Le Q V. EfficientDet: Scalable and Efficient Object Detection[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2020: 10781-10790.

三、端到端系统与边缘视觉案例

[7] Ju Y, Gu J. A Systolic Neural CPU Processor Combining Deep Learning and General-Purpose Computing With Enhanced Data Locality and End-to-End Performance[J]. IEEE Journal of Solid-State Circuits, 2023, 58(1): 215-233.

[10] Karnik T, Borkar N, Akella S, et al. A cm-scale self-powered intelligent and secure IoT edge mote featuring an ultra-low-power SoC in 14nm tri-gate CMOS[C]//IEEE

International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers. 2018: 46-48.

[11] Eki R, Ito Y, Yamaguchi Y, et al. 9.6 A 1/2.3inch 12.3Mpixel with on-chip 4.97TOPS/W CNN processor back-illuminated stacked CMOS image sensor[C]//IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers. 2021: 154-156.

[12] Genc H, Kim S, Amid A, et al. Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration[C]//Proceedings of the 58th ACM/IEEE Design Automation Conference. 2021: 769-774.

[13] Sun Y, Zhang Y, Wang H, Guo J, Zheng J, Ning H. SES-YOLOv8n: automatic driving object detection algorithm based on improved YOLOv8[J]. Signal, Image and Video Processing, 2024. doi:10.1007/s11760-024-03003-9.

[14] Li J, Cheang C F, Du Z, Yu X, Tang S, Cheng Q. DDRN: DETR with dual refinement networks for autonomous vehicle object detection[J]. Scientific Reports, 2025.

[15] Qian M, Wang Y, Liu S, Xu Z, Ji Z, Chen M, Wu H, Zhang Z. Real time wire rope detection method based on Rockchip RK3588[J]. Scientific Reports, 2025.

计划版本：*v4.0* 制定日期：*2026-03-11* 答辩截止：*2027* 年 *3* 月 目标平台：*Xilinx ZCU102*（*XCZU9EG-FFVB1156*）